Motion Recognition with Convolutional Neural Networks

Tuan Son Truong
Southern Methodist University · December 2024

Table of Contents

Abstract.....2

Introduction.....2

Literature Survey.....3

Approach.....3

Result.....7

Concluding Remark.....9

References.....10

Abstract

Training computers to recognize human motion has been one of the many applications of machine learning. However, a large part of the research has been focused on computer vision or the use of multiple sensors on different body parts to detect simple human motion. This praxis explores a different approach to motion recognition using only smartphone sensors. The objective is to classify motions into two categories: bringing the phone to eye level (yes motion) or other movements (no motion). Accelerometer and gyroscope data were collected over 5 seconds intervals to train a 12 layers Convolutional Neural Network (CNN) to perform this task. The results demonstrate the model's effectiveness in distinguishing intentional motions from background activities with an accuracy of 84.67% ± 3.32%.

Introduction

In today's world, smartphones have become an integral part of daily life. According to a 2023 survey by Reviews.org, the average American checks their phone 144 times per day. This frequent interaction has driven smartphone designs to prioritize convenience and ease of use. One such convenience that often goes unnoticed by the user but is sorely missed when absent is the smartphone's ability to recognize when the user wants to interact with it. This capability is not only necessary for the user’s experience but also conserves power by ensuring the display turns on only when needed.

Given the high frequency of phone usage, it is crucial for smartphones to reliably distinguish between the intentional motion of being brought to a user's face and random background movements. Typically, users raise their phones to eye level when interacting with them, making this motion a clear indicator to activate the display. However, as smartphones frequently experience various movements it becomes essential to differentiate these background motions from the deliberate action of bringing the phone to the face.

To address this challenge, my praxis utilizes machine learning to train a model capable of categorizing motion into two distinct categories: movements where the phone is brought to the user's face and those that are unrelated. The primary goal is to accurately identify intentional interactions while filtering out random background motion, ensuring the phone activates only when needed by the user.

Literature Survey

Training machine learning models to recognize human motion is a field known as Human Activity Recognition (HAR). Previous HAR studies, like the one done by Vidya and Sasikumar, have primarily focused on the categorization of a wide variety of simple motions such as sitting, walking, running, etc. To fully encapsulate these movements, Vidya and Sasikumar record their data using multiple sensors attached to different parts of the person's body and a smartphone. Because the goal of the praxis is to train the phone to do binary categorization to see if a person is using their phone or not, I will only utilize my phone to capture my data.

Modern smartphones come equipped with a wide range of sensors. However, utilizing multiple sensors simultaneously can significantly increase energy consumption, which is a critical concern for low-power devices like smartphones. Given the goal of this praxis, to continuously monitor whether the user intends to use the phone, it is essential to balance sensor usage for accuracy while minimizing energy consumption. According to studies by Shoaib et al., the accelerometer and gyroscope are well-suited to play primary roles in recognizing simple motions. Although the study has shown that combining these sensors may not always yield a noticeable improvement in overall HAR performance. However, I will still use both in tandem, as each excels in capturing distinct aspects of a more complicated movement, such as bringing a phone to eye level.

Studies on Human Activity Recognition (HAR) often utilize a wide range of machine learning models. One such study done by Logacjov et al. seeks to compare the different machine learning models to find which one has the best performance. According to the study, Support Vector Machine (SVM) slightly outperformed other models, achieving the highest F1 score, a combination of precision and recall. However, according to a different study by Ramamurthy and Roy, surveying the trends of machine learning in HAR, SVM relies heavily on feature engineering and uses shallow learning techniques. These characteristics make them less suitable for tasks involving complex motions, such as bringing a phone to eye level. In contrast, Ramamurthy and Roy observed that deep learning methods, such as CNN, excel at recognizing complex motion without the need for extensive feature engineering. Therefore, CNN is a more compatible choice for achieving the praxis' goal.

Approach

Data Collection:

One significant challenge in training a machine model to recognize human motion is collecting data that allows the model to generalize effectively across individuals. As Ramamurthy and Roy point out, the probability distribution of accelerometer data is highly user

dependent. As a result, a model trained on data from one individual, even if it performs well for that person, may not yield reliable results for others. Given this challenge, along with the absence of a publicly available dataset specifically for the motion of bringing a phone to eye level, I opted to record my own data. This approach differs from other studies done using smartphones for HAR as it enables me to personalize the CNN model and provides the flexibility to expand the dataset if required.

To keep the class sizes reasonably balanced in my dataset, I categorized the data into two classes for binary classification: motions involving bringing the phone to eye level (yes data) and motions that don't (no data). Given the diverse ways smartphones are used daily, the dataset is also needed to capture a variety of realistic scenarios. As a result, I will divide my yes and no data into smaller scenarios, with each containing its own sub-scenario, with 20 recordings per sub-scenario—half performed with the right hand and the other half with the left. The data collection will be done through an iPhone 14 plus using the MATLAB mobile app sensor measurement feature. The phone will measure its acceleration and rotation in the x, y, and z directions using its built-in accelerometer and gyroscope with a frequency of 100Hz.

The "Yes Data" is divided into four main scenarios: Lying, Sitting, Standing, and Stairs, with each scenario containing sub-scenarios—220 recording total.

1. **Lying**:
    - Bringing the phone to eye level starting from near the head.
    - Bringing the phone to eye level starting from near the side.
2. **Sitting**:
    - Raising the phone when it face-up on a table.
    - Flipping the phone when it face-down on a table.
3. **Standing**:
    - Bringing the phone to eye level while stationary.
    - Bringing the phone to eye level while walking.
    - Bringing the phone to eye level while running.
4. **Stairs**:
    - Bringing the phone to eye level while walking up the stairs.
    - Bringing the phone to eye level while running up the stairs.
    - Bringing the phone to eye level while walking down the stairs.
    - Bringing the phone to eye level while running down the stairs.

Because the "No data" encompasses a wide variety of motions, there are many ways to record it. To avoid having more "No Data" then "Yes Data" and creating bias during training, I prioritized no motions that start similarly to the yes motion, such as grabbing the phone from a table, bringing it to the ear, or raising it at an angle away from eye level.

The "No Data" is divided into three main scenarios: Grabbing, Standing, and Stairs, with each scenario containing sub-scenarios—180 recording total.

1. **Grabbing**:
    - Grabbing the phone when it is face-up on a table
    - Grabbing the phone when it is face-down on a table
2. **Standing**:
    - Performing the motion while stationary.
    - Performing the motion while walking.
    - Performing the motion while running.
    - For each motion:
        - First 10 recordings: Bringing the phone to the right and left ear
        - Last 10 recordings: Bringing the phone to the face at an angle away from eye level with right and left hand
3. **Stairs**:
    - Performing the motion while walking up the stairs.
    - Performing the motion while running up the stairs.
    - Performing the motion while walking down the stairs.
    - Performing the motion while running down the stairs.
    - For each motion:
        - First 10 recordings: Bringing the phone to the right and left ear
        - Last 10 recordings: Bringing the phone to the face at an angle away from eye level with right and left hand

Data preprocessing:

As for the testing data, I decided to record an additional 30 data sets, evenly split between "yes" and "no" data. Unlike the training and validation datasets, the testing data includes more varied and non-uniform background motions, such as transitioning from walking to running while bringing the phone to eye level, turning to one side while lying down, or walking and running down a spiral stair case. These scenarios introduce additional noise compared to the training and validation data, providing a more challenging set of samples for the CNN to classify accurately.

Before the data can be used in the CNN, it must first undergo preprocessing. This process involves standardizing the size of all recordings. Since most recordings slightly exceed 5 seconds, the data is trimmed to 500 samples. The accelerometer and gyroscope data are then combined into a single matrix with dimensions of 500x6x1, where the first three columns represent accelerometer data, and the last three columns represent gyroscope data. Because accelerometer readings tend to have larger magnitudes than gyroscope readings, this imbalance could lead to faster convergence for the accelerometer data compared to the gyroscope data. To prevent this, all data is scaled to the range of $[-1,1]$, ensuring balanced learning across features. Finally, the data is then assigned a label: "1" if the motion involves bringing the phone to eye

level, and "0" if it does not. To achieve a 1:4 ratio of validation to training data, the 1st, 10th, 11th, and 20th recordings are allocated to the validation dataset, while the remaining recordings are used for training.

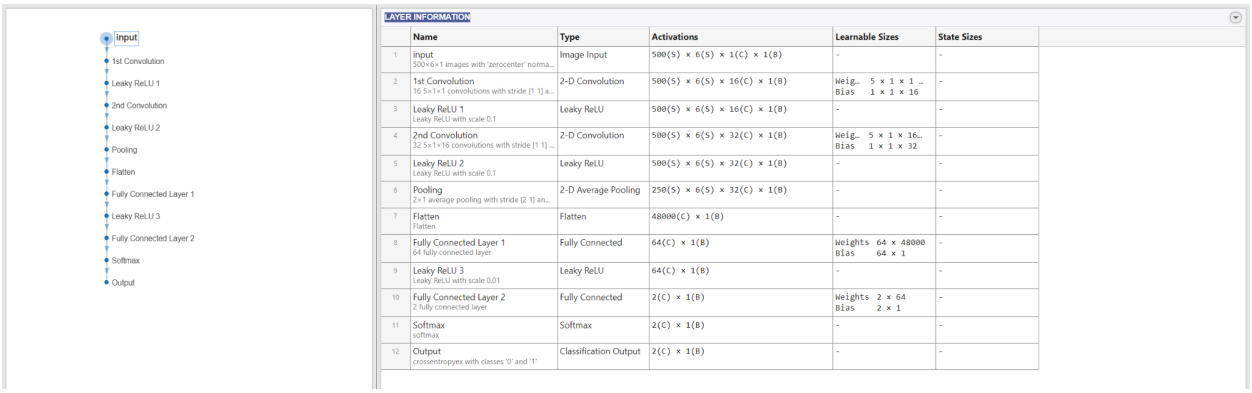
Implementation of CNN:

In a CNN, stacking convolutional layers enables the network to capture low-level features in earlier layers and more complex features in later layers. After each convolutional layer, Logacjov et al. recommend using a ReLU activation function to introduce non-linearity, enabling the network to learn complex patterns. However, standard ReLU sets negative activations to zero, which can cause some neurons to become inactive during training. To mitigate this, I chose to use Leaky ReLU, which retains a small, nonzero slope for negative activations. This allows gradients to propagate through negative regions, reducing the risk of inactive neurons. Additionally, He Normal initialization is used to initialize the convolutional layer weights, helping maintain stable signal and gradient magnitudes during training.

To further reduce the dimensionality of the feature maps while preserving their general patterns, an average pooling layer is used. After pooling, the resulting feature maps are flattened and passed through two fully connected layers. The second fully connected layer produces two outputs corresponding to the two motion classes. Finally, a Softmax layer converts these outputs into class probabilities for binary classification.

According to Logacjov et al., the optimal number of convolutional layers is four. However, given the smaller size of my personalized dataset, I found that using two convolutional layers is sufficient to achieve high training and validation accuracy while maintaining low training and validation time. Both convolutional layers use a 5×1 filter size with a stride of 1 and zero-padding to preserve the spatial dimensions of the input. The first convolutional layer receives an input matrix of 500×6×1 and applies 16 filters, producing 16 feature maps of size 500×6. These outputs are then passed to the second convolutional layer, which applies 32 filters to produce 32 feature maps of size 500×6. The average pooling layer uses a filter size of 2×1 with a vertical stride of 2 and a horizontal stride of 1. This reduces the number of rows by half, resulting in an output of 250×6×32. The pooled output is then flattened into a 1×48,000 vector and passed to a fully connected layer with 64 neurons, followed by another fully connected layer with 2 neurons representing the two motion classifications. Finally, a SoftMax layer converts the outputs into class probabilities for binary classification.

Figure 1: CNN architecture



LAYER INFORMATION

	Name	Type	Activations	Learnable Sizes	State Sizes
1	input 500×6×1 images with 'zerocenter' norma...	Image Input	500(S) × 6(S) × 1(C) × 1(B)	-	-
2	1st Convolution 16 5×1×1 convolutions with stride [1 1] a...	2-D Convolution	500(S) × 6(S) × 16(C) × 1(B)	Weig... 5 × 1 × 1 ... Bias 1 × 1 × 16	-
3	Leaky ReLU 1 Leaky ReLU with scale 0.1	Leaky ReLU	500(S) × 6(S) × 16(C) × 1(B)	-	-
4	2nd Convolution 32 5×1×16 convolutions with stride [1 1] ...	2-D Convolution	500(S) × 6(S) × 32(C) × 1(B)	Weig... 5 × 1 × 16... Bias 1 × 1 × 32	-
5	Leaky ReLU 2 Leaky ReLU with scale 0.1	Leaky ReLU	500(S) × 6(S) × 32(C) × 1(B)	-	-
6	Pooling 2×1 average pooling with stride [2 1] an...	2-D Average Pooling	250(S) × 6(S) × 32(C) × 1(B)	-	-
7	Flatten Flatten	Flatten	48000(C) × 1(B)	-	-
8	Fully Connected Layer 1 64 fully connected layer	Fully Connected	64(C) × 1(B)	Weights 64 × 48000 Bias 64 × 1	-
9	Leaky ReLU 3 Leaky ReLU with scale 0.01	Leaky ReLU	64(C) × 1(B)	-	-
10	Fully Connected Layer 2 2 fully connected layer	Fully Connected	2(C) × 1(B)	Weights 2 × 64 Bias 2 × 1	-
11	Softmax softmax	Softmax	2(C) × 1(B)	-	-
12	Output crossentropyex with classes '0' and '1'	Classification Output	2(C) × 1(B)	-	-

For backpropagation and weight updates, I selected stochastic gradient descent with a step size of 0.01 and 25 epochs. The choice of 25 epochs was based on observing that the training accuracy reaches 100%, while the validation accuracy stabilizes at 97.5%. As shown in the table below, increasing the number of epochs beyond 25 does not result in any significant improvement and only increases the training time for the CNN.

Table 1: Training Progress for 100 epochs

```
Training on single CPU.
Initializing input data normalization.
|=========================================================================================================================|
|  Epoch  |  Iteration  |  Time Elapsed  |  Mini-batch  |  Validation  |  Mini-batch  |  Validation  |  Base Learning  |
|         |             |   (hh:mm:ss)   |   Accuracy   |   Accuracy   |     Loss     |     Loss     |      Rate       |
|=========================================================================================================================|
|       1 |           1 |       00:00:01 |       50.00% |       87.50% |       0.7542 |       0.3999 |          0.0100 |
|      25 |          50 |       00:00:16 |      100.00% |       97.50% |       0.0654 |       0.1275 |          0.0100 |
|      50 |         100 |       00:00:31 |      100.00% |       97.50% |       0.0236 |       0.0988 |          0.0100 |
|      75 |         150 |       00:00:47 |      100.00% |       97.50% |       0.0127 |       0.0955 |          0.0100 |
|     100 |         200 |       00:01:02 |      100.00% |       97.50% |       0.0082 |       0.0956 |          0.0100 |
|=========================================================================================================================|
Training finished: Max epochs completed.
```

Result

Due to the random initialization of weights of the CNN, each training session results in a slightly different model with varying levels of accuracy. To comprehensively evaluate the performance of my CNN, I will run the training process 20 times and calculate the average training, validation, and testing accuracies. However, to accurately gauge the effectiveness of my CNN model at binary classification, I will also compute the precision, recall, and F1 score of the model for the testing data.

As shown in the table, the training accuracy for all trials consistently reaches 100%. However, the validation accuracy averages around 97.63%, suggesting that the CNN is slightly overfitting and performs slightly worse on unseen data. Moreover, the testing data achieves an

average accuracy of only 84.67%. The model's average precision of 83.89% indicates that it reliably identifies the motion of picking up the phone. Additionally, the average recall of 88.44% demonstrates that the model successfully captures most instances of the picking-up phone motion, with relatively few missed cases. The lower testing accuracy, relative to the training and validation accuracy, may be attributed to increased noise in the testing data compared to the training and validation datasets.

Table 2: CNN performance over 20 trials

Trial	Training Accuracy (%)	Validaiton Accuracy (%)	Testing Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
1	100	97.50	83.33	82.35	87.50	84.85
2	100	97.50	86.67	87.50	87.50	87.50
3	100	97.50	90.00	88.24	93.75	90.91
4	100	96.25	90.00	88.24	93.75	90.91
5	100	96.25	83.33	82.35	87.50	84.85
6	100	98.75	83.33	82.35	87.50	84.85
7	100	98.75	80.00	81.25	81.25	82.25
8	100	97.50	90.00	93.33	87.50	90.32
9	100	98.75	83.33	82.35	87.50	84.85
10	100	98.75	86.67	83.33	93.75	88.24
11	100	96.25	80.00	75.00	93.75	83.33
12	100	98.75	86.67	83.33	93.75	88.24
13	100	93.75	86.67	83.33	93.75	88.24
14	100	98.75	83.33	82.35	87.50	84.85
15	100	98.75	83.33	82.35	87.50	84.85
16	100	98.75	80.00	85.71	75.00	80.00
17	100	97.50	83.33	82.35	87.50	84.85
18	100	97.50	86.67	83.33	93.75	88.24
19	100	97.50	80.00	81.25	81.25	81.25
20	100	97.50	86.67	87.50	87.50	87.50

Table 3: Average CNN results

Average Training Accuracy (%)	100 ± 0
Average Validaiton Accuracy (%)	97.63 ± 1.28
Average Testing Accuracy (%)	84.67 ± 3.32
Average Precision (%)	83.89 ± 5.08
Average Recall (%)	88.44 ± 5.08
Average F1 Score (%)	86.04 ± 3.08

Compared to the CNN evaluated by Logacjov et al., my model uses fewer convolutional layers (2 instead of 4) and fewer training epochs (25 instead of 80). Despite its simpler architecture, my CNN achieved an average testing accuracy of 84.67%. However, Logacjov et al. focused on multi-class human activity recognition, whereas my project focuses on binary classification of a specific phone gesture. Because the datasets and classification objectives differ, the performance results are not directly comparable.

Concluding Remark

In this praxis, we explored the approach of using CNN for human activity recognition by focusing on the binary classification of a complex motion: bringing a phone to eye level. Unlike traditional methods that rely on multiple sensors, our approach utilizes only the accelerometer and gyroscope data from a single smartphone, our smartphone. Although the CNN model performed worse on the testing data compared to the training and validation data, the model still tested with an accuracy of 84.67%, along with its average precision of 83.89% and recall of 88.44%. As a result, it demonstrates that our CNN model has the ability to distinguish intentional phone motions from background activity.

To better improve the reliability of my CNN model and align the testing accuracy with the training and validation accuracy, several improvements can be made. First, incorporating noisier data, such as transitions between multiple background motions or scenarios like walking up a spiral staircase instead of a straight one, could help the model generalize better. Additionally, training and validation data can be diversified by including different ways people bring their phones to their faces, such as varying angles of motion. To ensure that the model performs well regardless of the user, data should be collected from multiple individuals. Lastly, for practical application, the duration of input data needs to be reduced. Currently, my CNN requires 5 seconds of data for classification, which delays the phone's ability to turn on instantly when raised. Reducing this to 1 second would enable faster response times, though it would require additional effort to process and optimize the data.

References

[1] A. Logacjov, K. Bach, A. Kongsvold, H. B. Bardstu, and P. J. Mork, “HARTH: A Human Activity Recognition Dataset for Machine Learning,” *Sensors (Basel, Switzerland)*, vol. 21, no. 23, pp. 7853-, 2021, doi: 10.3390/s21237853.

[2] B. Vidya and S. P, “Wearable multi-sensor data fusion approach for human activity recognition using machine learning algorithms,” *Sensors and actuators. A. Physical.*, vol. 341, pp. 113557-, 2022, doi: 10.1016/j.sna.2022.113557.

[3] S. Ramasamy Ramamurthy and N. Roy, “Recent trends in machine learning for human activity recognition—A survey,” *Wiley interdisciplinary reviews. Data mining and knowledge discovery*, vol. 8, no. 4, pp. e1254-n/a, 2018, doi: 10.1002/widm.1254.

[4] M. Shoaib, S. Bosch, O. D. Incel, H. Scholten, and P. J. M. Havinga, “Fusion of Smartphone Motion Sensors for Physical Activity Recognition,” *Sensors (Basel, Switzerland)*, vol. 14, no. 6, pp. 10146–10176, 2014, doi: 10.3390/s140610146.

[5] *Reviews.org*, “Cell phone addiction statistics: The definitive guide 2024,” *Reviews.org*. [Online]. Available: https://www.reviews.org/mobile/cell-phone-addiction/.